VoronoiSCAN: Enhancing DBSCAN with Voronoi Tessellation for Distributed Clustering

Supplementary Material

1 Internal DBSCAN

The pseudocode of the DBSCAN formulation of Gan and Tao [4] with the parallel improvements of Wang, Gu, and Shun [9] is provided in Algorithm 1. Given a set of points $\mathcal{P}$, ϵ and `minPts` the algorithm creates a hypercube grid. Each grid cell is initialized with a side length of $\epsilon/\sqrt{d}$ (Line 1) and the core points $\mathcal{F}$ are initialized as an empty set (Line 2). If a grid cell contains at least `minPts` points, it is called core cell and all points in that cell can be marked as core points in parallel (Lines 4–6). Otherwise, for each point p_x in that cell, we count in parallel the points in ϵ range in all the neighboring cells and re-evaluate if p_x is a core point (Lines 8–13). Because clusters can span multiple grid cells, we need to connect multiple core grid cells. For this, we use a *Union-Find* data structure [3]. For each unlinked pair of adjacent core grid cells, we compute the *bichromatic closest pair* [1] in parallel to check if there is a pair of core points, which are within ϵ distance, in order to link those cells (Lines 14–20). For all points in the core cells, we obtain the cluster label from the *Union-Find* data structure (Lines 22–24). As a last step, potential border points in non-core cells are assigned to the existing clusters by iterating in parallel over all non-core points in each cell to find core points within ϵ distance in the neighbor grid cells (Lines 25–31).

2 Correctness of VoronoiSCAN

2.1 Proofs

In the following, we prove that *VoronoiSCAN* produces results equivalent to DBSCAN through a series of lemmas. Let $\mathcal{P}$ be the dataset, ϵ be the neighborhood radius, and `minPts` be the minimum points parameter.

Lemma 1 (Path Existence and Point Inclusion) *Let $p_x \in v_j$ be a point within ϵ-distance of cell v_i. Then, the Assignment algorithm ensures $p_x \in v_i^+$.*

Algorithm 1: Local DBSCAN

Input: $\mathcal{P}$: set of points, ϵ: radius, minPts: density threshold
Output: C: cluster labels

$\mathcal{G} \leftarrow$CELLS$(\mathcal{P}, \epsilon)$
$\mathcal{F} \leftarrow \emptyset$
foreach $g \in \mathcal{G}$ **do in parallel**
  if $|g| \geq minPts$ **then**
    foreach $p_i \in g$ **do**
      $\mathcal{F} \leftarrow \mathcal{F} \cup \{p_i\}$
  else
    foreach $p_i \in g$ **do in parallel**
      $c \leftarrow |g|$
      foreach $h \in g$.NEIGHBORCELLS(ϵ) **do**
        $c \leftarrow c + h$.RANGECOUNT(p_i, ϵ)
      if $c \geq minPts$ **then**
        $\mathcal{F} \leftarrow \mathcal{F} \cup \{p_i\}$
$uf \leftarrow$UNIONFIND()
foreach $g \in \mathcal{G} : |g| \geq minPts$ **do in parallel**
  foreach $h \in g$.NEIGHBORCELLS$(\epsilon) : |h| \geq minPts$ **do**
    if $g > h \wedge uf$.FIND$(g) \neq uf$.FIND(h) **then**
      $(p_x, p_y) \leftarrow$BCP(g, h)
      if $d(p_x, p_y) \leq \epsilon$ **then**
        uf.LINK(g, h)
$C \leftarrow [0]_{i=1}^{|\mathcal{P}|}$
foreach $g \in \mathcal{G} : |g| \geq minPts$ **do in parallel**
  foreach $p_x \in g : p_x \in \mathcal{F}$ **do in parallel**
    $C[p_x] \leftarrow uf$.FIND(g)
foreach $g \in \mathcal{G} : |g| < minPts$ **do in parallel**
  foreach $p_x \in g : p_x \notin \mathcal{F}$ **do in parallel**
    foreach $h \in g \cup g$.NEIGHBORCELLS(ϵ) **do**
      foreach $p_y \in h : p_y \in \mathcal{F}$ **do in parallel**
        if $d(p_x, p_y) \leq \epsilon$ **then**
          $C[p_x] \leftarrow C[p_y]$
          break
return C

1. *There exists a path of Voronoi cells $S(v_i, v_{i+1}, \ldots, v_j)$ in the Delaunay graph N_G such that p_x is within ϵ-distance of all cells in the path.*

2. *The Assignment algorithm ensures $p_x \in v_i^+$ by iteratively moving p_x along this path.*

Proof 1 *The cells v_i and v_j are connected through a sequence of Voronoi cells $S(v_i, v_{i+1}, \ldots, v_j)$ in the Delaunay graph G such that p_x is within ϵ-distance of all cells in the path. We proceed by with an induction on the path length $|S|$.*

1. *If $|S| = 2$, then v_i and v_j are direct neighbors, i.e. $v_j \in N_G(v_i)$. The expanded cell v_i^+ is defined by the adjusted centroids of its direct neighbors:*

$$\hat{z}_j^i = z_j + 2\epsilon \frac{z_j - z_i}{\|z_j - z_i\|_2}$$

Let k be the current iteration of the algorithm and

$$\hat{\mathcal{P}}_i^k = \begin{cases} \bigcup\limits_{v_j \in N_G(v_i)} \{p_x \in v_j | d(z_j, p_x) \geq \frac{\min\limits_{v_k \in N_G(v_j)} d(z_k, z_j)}{2} - \epsilon\} & k = 0 \\ \bigcup\limits_{v_j \in N_G(v_i)} \hat{\mathcal{P}}_j^{k-1} \setminus \mathcal{V}_i & \text{otherwise} \end{cases}$$

By construction of the extended Voronoi cell v_j^+, $p_x \in \hat{\mathcal{P}}_j^k$ and, hence, $p_x \in v_i^+ \iff d(p_x, z_i) < d(p_x, \hat{z}_j^i)$. Therefore, any $p_x \notin v_i$ within ϵ-distance of v_i will be part of v_i^+ and be marked as stolen in $\hat{\mathcal{P}}_i^{k+1}$ for the next iteration.

2. *If $|S| > 2$, then S can be decomposed in $S = S_1(v_i, \ldots, v_{j-1}) \circ S_2(v_{j-1}, v_j)$. In S_2, v_{j-1} and v_j are direct neighbors and, hence, can be resolved by case (1) such that p_x has been stolen by v_{j-1} in the previous iteration $k-1$, i.e. $p_x \in \hat{\mathcal{P}}_{j-1}^{k-1}$. Consequently, $p_x \in \hat{\mathcal{P}}_{j-2}^k$ for the current iteration k and cell v_{j-2}.*

By recursively unfolding this sequence, the inductions completes, concluding the proof.

Lemma 2 (Neighborhood Completeness) *For any point $p_x \in \mathcal{P}$ there exists a Voronoi cell v_i, such that $N_\epsilon^{i^+}(p_x)$ in* VoronoiSCAN *is identical to the ϵ-neighborhood $N_\epsilon(p_x)$ in DBSCAN.*

Proof 2 *Let $\mathcal{Z} = \{z_1, z_2, \ldots, z_n\}$ be the set of centroids defining Voronoi cells $v_i = \{p_x \in \mathcal{P} \mid d(p_x, z_i) < d(p_x, z_j)\ \forall z_j \neq z_i\}$.*

Neighborhood Equivalence *Now we prove that for any $p_x \in v_i$, $N_\epsilon^{i^+}(p_x) = N_\epsilon(p_x)$ by considering all possible locations of points $p_y \in \mathcal{P}$ where $d(p_x, p_y) \leq \epsilon$:*

1. *If $p_y \in v_i$: Then $p_y \in N_\epsilon^{i^+}(x)$ by construction of the local DBSCAN.*

2. *If $p_y \in v_j$ for any other cell v_j: By Lemma 1 we know that $p_y \in v_i^+$. Therefore $p_y \in N_\epsilon^{i^+}(p_x)$.*

Lemma 3 (Core Point Equivalence) *A point $p_x \in \mathcal{P}$ with an associated Voronoi cell v_i is a core point in* VoronoiSCAN *if and only if it is a core point in DBSCAN.*

Proof 3 *A point p_x is a core point in DBSCAN if $|N_\epsilon(p_x)| \geq$ `minPts`. From Lemma 2, we know that $N_\epsilon^{i^+}(p_x) = N_\epsilon(p_x)$. Therefore:*

$$|N_\epsilon^{i^+}(p_x)| \geq \texttt{minPts} \iff |N_\epsilon(p_x)| \geq \texttt{minPts}$$

Thus, the core point classification is equivalent in both algorithms.

Lemma 4 (Global Density-Connectivity) *Two points p_x and $p_y \in \mathcal{P}$ are density-connected in* VoronoiSCAN *if and only if they are density-connected in DBSCAN.*

Proof 4 *($\Rightarrow$) Let p_x, p_y be density-connected in DBSCAN. It is to be shown that p_x and p_y are then also density-connected in* VoronoiSCAN*. By definition of density-connectivity, there exists a sequence of points $P(p_1, \ldots, p_n)$ with $p_1, \ldots, p_n \in \mathcal{P}$, $p_1 = p_x$ and $p_n = p_y$, where p_1 and p_n might be border points, and a sequence of cells $S(v_1, \ldots, v_n)$ such that for each point p_m, there exists a local cluster $c_m(p_m)$ in some cell v_m containing p_m. For consecutive points p_m, p_{m+1}:*

1. *If $|S| = 2$, then v_m and v_{m+1} are neighbors in N_G. Here, we have to distinguish two cases:*
   a) *If p_m and p_{m+1} are in the same cell, i.e., $v_m = v_{m+1}$, then p_m is a core point and, therefore, p_{m+1} is directly density-reachable from p_m by Lemma 2 and 3. Therefore, $c_m(p_m) = c_m(p_{m+1})$ and, hence, p_m and p_{m+1} must be density-connected in* VoronoiSCAN *as well.*
   b) *If p_m and p_{m+1} are in different cells, i.e., $v_m \neq v_{m+1}$, we have to consider p_{m+1}: If p_{m+1} is a border point, then $p_{m+1} \in N_\epsilon^{m^+}(p_m)$. Therefore, p_m and p_{m+1} are directly-density reachable and, hence, also density connected. If p_{m+1} is a core point, then $p_m, p_{m+1} \in \mathcal{M}_m^{m+1}$. Consequently, $(p_m, p_{m+1}) \in E_D^{m,m+1}$. Then, p_m and p_{m+1} are in the same connected component in $G_D^{m,m+1} = (\mathcal{M}_m^{m+1}, E_D^{m,m+1})$ and, hence, their local clusters are connected due to $(c_m(p_m), c_{m+1}(p_{m+1})) \in E_C$ in $G_C = (V_C, E_C)$.*
2. *If $|S| > 2$, then there is a sequence of cells $S(v_m, ..., v_{m+1})$ with consecutive cells being neighbors. We can decompose S as $S(v_m, v_k, \ldots, v_{m+1}) = S_1(v_m, v_k) \circ S_2(v_k, \ldots, v_{m+1})$. v_k is then a shared neighbor of v_m and v_{k+1} with $p_m, p_{m+1} \in v_k^+$. S_1 can be resolved by case 1b with an additional edge $(c_m(p_m), c_k(p_{m+1})) \in E_C$. Note that for the first and the last decomposition of S case 1a might be applied. For any other decomposition of S, an edge $(c_k(p_m), c_{k+1}(p_{m+1})) \in E_C$ is created. By unfolding this decomposition for S_2 there will be a path between $c_m(p_m)$ and*

$c_{m+1}(p_{m+1})$ *in* G_C*. Consequently,* $c_m(p_m)$ *and* $c_{m+1}(p_{m+1})$ *will be part of the same connected component in* G_C*. Because the local clusters for each connected component in* G_C *are merged into the same final cluster, the points in these local clusters must also be density-connected in DBSCAN.*

(⇐) Let p_x, p_y *be density-connected in* VoronoiSCAN*. This implies there exists a path of local clusters* $c_1, \ldots, c_k$ *in the cluster graph* G_C*, where* $p_x \in c_1$ *and* $p_y \in c_k$*. For any adjacent pair* c_m, c_{m+1} *such that* $d(p_m, p_{m+1}) \leq \epsilon$*:*

1. *If* $c_m = c_{m+1}$*, then* p_m *and* p_{m+1} *are in the same local cluster in an extended cell* v_m^+*, Lemma 2 ensures* v^+ *contains the complete* ϵ*-neighborhood for all internal core points. Thus, by Lemma 3,* p_m *is a core point and, since* $d(p_m, p_{m+1}) \leq \epsilon$*,* p_{m+1} *is directly density-reachable from* p_m *and they are in same cluster in DBSCAN.*

2. *If* $c_m \neq c_{m+1}$*, we know that* p_m *and* p_{m+1} *are in different cells* v_m *and* v_{m+1}*. Because there then exists an edge* $(c_m, c_{m+1}) \in E_C$ *there must be a path* $\mathcal{P}(p_m, \ldots, p_{m+1})$ *for* p_m *and* p_{m+1} *in* $G_D^{m,m+1}$ *through the merge points* $\mathcal{M}_m^{m+1}$ *in the boundary overlap* $\mathcal{E}_m^{m+1}$*, i.e.* $(p_m, p_{m+1} \in E_D^{m,m+1})$*. This means that along this path* P *each consecutive points have a distance of* $\leq \epsilon$*.*

By transitivity, the concatenation of these valid intra-cluster segments and inter-cluster bridges forms a continuous density-connected chain in DBSCAN.

2.2 Comparison to sequential DBSCAN

We further evaluated the clustering performance of VoronoiSCAN in comparison with the sequential DBSCAN variant. For this, we subsampled the synthetic *densired* datasets of dimensionality d to 100,000 data points and adopted the cluster labels obtained from sequential DBSCAN as ground-truth annotations. We then generated multiple clusterings by varying ϵ and `minPts`, and, in the case of VoronoiSCAN, by additionally varying the number of Voronoi cells. For each configuration, we computed the Normalized Mutual Information (NMI) between the ground-truth labels and the labels produced by VoronoiSCAN. The NMI scales between 0 (no mutual information) and 1 (perfect correlation). The results, summarized in Table 1, indicate that VoronoiSCAN reproduces the clusterings of sequential DBSCAN with high accuracy. It is important to note that DBSCAN is not fully deterministic in the assignment of border points, as this assignment depends on the order in which they are processed. This non-determinism accounts for the fact that, for $d = 2$, the NMI is not exactly 1.0 but still > 0.99 such that *VoronoiSCAN* can be considered as exact.

3 Complexity Analysis of VoronoiSCAN

In this section, we provide a complexity analysis of the different steps in VoronoiSCAN.

Table 1: NMI of VoronoiSCAN compared to sequential DBSCAN as ground truth for different clusterings.

			d			
`minPts`	ϵ	Num. Cells	2	3	4	5
6	0.015	16	0.998	1.000	1.000	1.000
10	0.020	16	0.999	1.000	1.000	1.000
14	0.025	16	0.999	1.000	1.000	1.000
6	0.015	32	0.991	1.000	1.000	1.000
10	0.020	32	0.999	1.000	1.000	1.000
14	0.025	32	1.000	1.000	1.000	1.000
6	0.015	64	0.999	1.000	1.000	1.000
10	0.020	64	0.998	1.000	1.000	1.000
14	0.025	64	0.997	1.000	1.000	1.000

3.1 Partitioning

Sampling To fetch n samples from the dataset, we seek to a random byte-offset of the input file each to select a row. Therefore, the runtime complexity of the sampling process is bound by $\mathcal{O}(n)$.

Delaunay graph construction & Voronoi tessellation with ϵ-boundary Delaunay graph construction takes $\mathcal{O}(n \log n)$ time in $\mathbb{R}^2$ [5] and $\mathcal{O}(n^{\lceil d/2 \rceil})$ in higher dimensions [7]. To compute the adjusted centroids $\hat{z}_j^i$ we iterate over the graph edges, performing constant-time vector operations for each neighbor pair. Thus, the complexity of the expansion step is linear in the number of edges—i.e., $\mathcal{O}(n)$ in 2D and bounded by $\mathcal{O}(n^{\lceil d/2 \rceil})$ generally. Because the centroid set is small relative to the dataset ($n \ll |\mathcal{P}|$), this precomputation is highly efficient for low-dimensional spaces.

Cell assignment With an index structure, such as a kd-tree, z_i look-up queries are bounded by $\mathcal{O}(\log n)$ [2]; hence, Algorithm 1 (see paper) has an overall runtime complexity in $\mathcal{O}(|\mathcal{P}| \log n)$, which can be reduced to $\mathcal{O}(\frac{|\mathcal{P}|}{m} \log n)$ with parallelization on m processors. Because $n \ll |\mathcal{P}|$, the assignment remains efficient.

For Algorithm 2 (see paper) the iterative evaluation of the points in the sets $\hat{\mathcal{P}}_i^k$ constitutes the primary computational cost. In each iteration k, the algorithm checks (parallelized by the Voronoi cell v_i) for each p_x of the sets $\hat{\mathcal{P}}_i^k$, whether or not it needs to be stolen. Given that d represents the average degree of a Delaunay graph with n cells, the number of points in the cells is balanced, and n also corresponds to the number of parallel threads in the system, we can approximate the number of points for each cell with $\frac{|\mathcal{P}|}{n}$. For each point $p_x \in \hat{\mathcal{P}}_i^k$, the algorithm performs d distance comparison in $\mathcal{O}(1)$ time. Thus, the complexity for one iteration is $\mathcal{O}(d \cdot \frac{|\mathcal{P}|}{n})$. Because the algorithm runs iteratively, we define $k_{\max}$ to be the number of iterations required for convergence; $k_{\max}$ depends on the structure of the Voronoi cells and the spatial distribution of the points.

In practice, $k_{\max}$ is typically small, because the number of points eligible for stealing diminishes rapidly with each iteration. Consequently, the total runtime complexity is given by $\mathcal{O}(k_{\max} \cdot d \cdot \frac{|\mathcal{P}|}{n})$.

3.2 Local DBSCAN

Although, in theory, the worst-case runtime complexity of DBSCAN is quadratic [4, 6], i.e., in the parallel setting considered here $\mathcal{O}(\max(|v_i^+|)^2)$, the use of spatial index structures substantially accelerates neighborhood queries. As a consequence, truly quadratic behavior is rarely observed in practice, and the effective complexity is $\Omega(|\mathcal{P}|^{4/3})$ [4]. Assuming that the $|\mathcal{P}|$ data points are evenly distributed across the n Voronoi cells, the worst-case runtime can then be approximated by $\mathcal{O}\big(\frac{|\mathcal{P}|^2}{n^2}\big)$, while in most practical scenarios it behaves closer to $\Omega\Big(\big(\frac{|\mathcal{P}|}{n}\big)^{4/3}\Big)$.

3.3 ϵ-Boundary Merge

The runtime of the ϵ-boundary merge is determined by the number of neighboring Voronoi cell pairs (v_i, v_j), given by the edges of the Delaunay graph $G(\mathcal{Z}, E)$, and the number of merge points in their shared ϵ-boundaries. For a neighboring pair $(v_i, v_j) \in E$ each $p_x \in \mathcal{M}_j^i$ is processed once, and the corresponding local cluster identifiers are united using a Union-Find [3] data structure. This yields a runtime of $\mathcal{O}(|\mathcal{M}_j^i| \cdot \alpha(\kappa_i + \kappa_j))$ per neighboring cell pair, where κ_i and κ_j denote the numbers of local clusters in v_i^+ and v_j^+ with $\alpha(\cdot)$ as the inverse Ackermann [8] function. Since the merges for different Delaunay edges are independent, they are executed in parallel, yielding near-linear scaling in the number of merge points in practice.

3.4 Global Cluster Merging

The runtime complexity is $\mathcal{O}(|E_C| \cdot \alpha(|V_C|))$ to compute the connected components and $\mathcal{O}(|\mathcal{P}|/n)$ for the parallel labeling of the n Voronoi cells. Therefore, the total runtime for this step is $\mathcal{O}(|E_C| \cdot \alpha(|V_C|) + |\mathcal{P}|/n)$.

References

[1] Pankaj K. Agarwal et al. "Euclidean minimum spanning trees and bichromatic closest pairs". In: *Proceedings of the sixth annual symposium on Computational geometry - SCG '90*. SCG '90. ACM Press, 1990, pp. 203–210. DOI: 10.1145/98524.98567.

[2] Jon Louis Bentley. "Multidimensional binary search trees used for associative searching". In: *Communications of the ACM* 18.9 (Sept. 1975), pp. 509–517. ISSN: 1557-7317. DOI: 10.1145/361002.361007.

[3] Bernard A. Galler and Michael J. Fisher. "An improved equivalence algorithm". In: *Communications of the ACM* 7.5 (May 1964). Disjoint-Set Union, pp. 301–303. ISSN: 1557-7317. DOI: 10.1145/364099.364331.

[4] Junhao Gan and Yufei Tao. "DBSCAN Revisited: Mis-Claim, Un-Fixability, and Approximation". In: *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. SIGMOD/PODS'15. ACM, May 2015. DOI: 10.1145/2723372.2737792.

[5] Geo Leach. "Improving worst-case optimal Delaunay triangulation algorithms". In: *4th Canadian Conference on Computational Geometry*. Vol. 2. Citeseer. 1992, p. 15.

[6] Erich Schubert et al. "DBSCAN Revisited, Revisited: Why and How You Should (Still) Use DBSCAN". In: *ACM Transactions on Database Systems* 42.3 (July 2017), pp. 1–21. ISSN: 1557-4644. DOI: 10.1145/3068335.

[7] Raimund Seidel. "The upper bound theorem for polytopes: an easy proof of its asymptotic version". In: *Computational Geometry* 5.2 (Sept. 1995), pp. 115–116. ISSN: 0925-7721. DOI: 10.1016/0925-7721(95)00013-y.

[8] Robert Endre Tarjan. "Efficiency of a Good But Not Linear Set Union Algorithm". In: *Journal of the ACM* 22.2 (Apr. 1975). Union-Find Sets, pp. 215–225. ISSN: 1557-735X. DOI: 10.1145/321879.321884.

[9] Yiqiu Wang, Yan Gu, and Julian Shun. "Theoretically-Efficient and Practical Parallel DBSCAN". In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (2019). DOI: 10.1145/3318464.3380582.